

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
Instructor(s) Name			
		Mr. S Naresh Kumar	
		Ms. B. Swathi	
		Dr. Sasanko Shekhar Gantayat	
		Mr. Md Sallauddin	
		Dr. Mathivanan	
		Mr. Y Srikanth	
		Ms. N Shilpa	
		Dr. Rishabh Mittal (Coordinator)	
		Dr. R. Prashant Kumar	
		Mr. Ankushavali MD	
		Mr. B Viswanath	
		Ms. Sujitha Reddy	
		Ms. A. Anitha	
		Ms. M.Madhuri	
		Ms. Katherashala Swetha	
		Ms. Velpula sumalatha	
Mr. Bingi Raju			
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week6 – Monday	Time(s)	23CSBTB01 To 23CSBTB52
NAME	M.Anu Chandra	BATCH	48
Assignment Number: 11.1(Present assignment number)/24(Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 11 – Data Structures with AI: Implementing Fundamental Structures Lab Objectives	Week6 - Monday	

- Use AI to assist in designing and implementing fundamental data structures in Python.
- Learn how to prompt AI for structure creation, optimization, and documentation.
- Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
- Enhance code quality with AI-generated comments and performance suggestions.

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

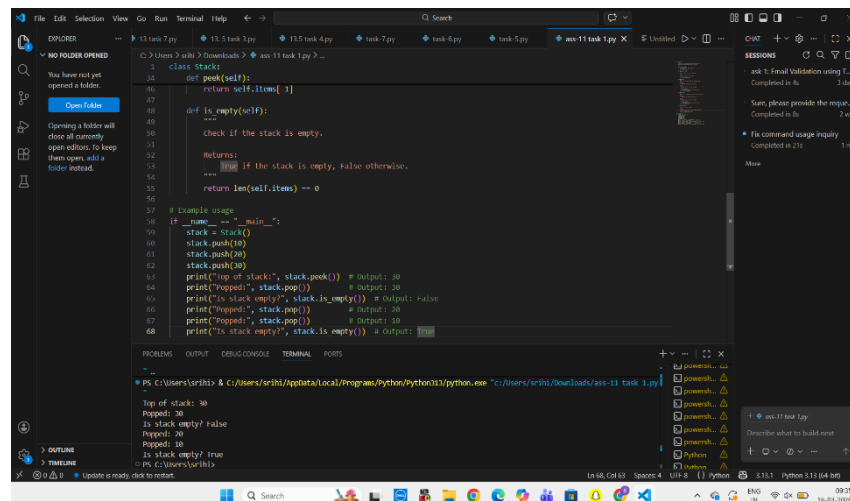
PROMPT : Generate a Python class named Stack that implements a stack data structure using a list.

Include the following methods: push(), pop(), peek(), and is_empty().

Follow the LIFO principle and add proper comments and docstrings explaining each method.

Also include a small example usage.

CODE :



```

1 class Stack:
2     def __init__(self):
3         self.items = []
4
5     def is_empty(self):
6         """
7         Check if the stack is empty.
8         Returns:
9             True if the stack is empty, False otherwise.
10        """
11        return len(self.items) == 0
12
13    # Example usage
14    if __name__ == "__main__":
15        stack = Stack()
16        stack.push(10)
17        stack.push(20)
18        stack.push(30)
19        print("Top of stack:", stack.peek()) # Output: 30
20        print("Popped:", stack.pop()) # Output: 30
21        print("is stack empty?", stack.is_empty()) # Output: False
22        print("Popped:", stack.pop()) # Output: 20
23        print("is stack empty?", stack.is_empty()) # Output: True
24
25    PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
26
27    PS C:\Users\srishi & C:\Users\srishi\AppData\Local\Programs\Python\Python111\python.exe "C:\Users\srishi\Downloads\act-11 task 1.py"
28
29    Top of stack: 30
30    Popped: 30
31    is stack empty? False
32    Popped: 20
33    Popped: 10
34    is stack empty? True
35    PS C:\Users\srishi
  
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

Explanation : In this task, I implemented the Stack data structure using Python. A stack follows the LIFO (Last In First Out) principle, which means the element that is inserted last will be removed first. First, I opened Visual Studio Code and created a new Python file. Then I defined a class called Stack. Inside the class, I created a

list to store the stack elements. After that, I implemented the main stack operations such as to insert an element, to remove the top element, to view the top element, to check whether the stack is empty. Finally, I tested the stack operations by inserting and removing elements.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

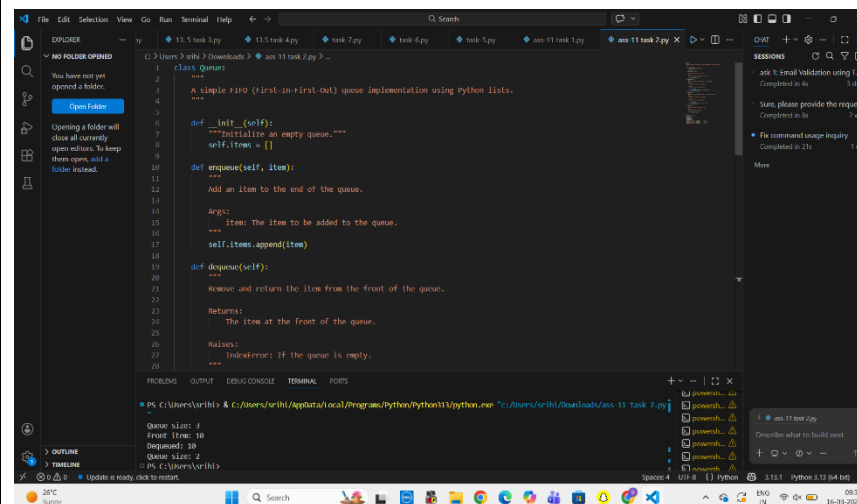
PROMPT : Write a Python class Queue that implements a FIFO queue using Python lists.

Include the methods enqueue(), dequeue(), peek(), and size().

Add comments and docstrings explaining the functionality of each method

and provide a short example demonstrating queue operations.

CODE :



```
1 class Queue:
2     """
3     A simple FIFO (first-in-first-out) queue implementation using python lists.
4     """
5     def __init__(self):
6         """Initialize an empty queue."""
7         self.items = []
8
9     def enqueue(self, item):
10        """
11        Add an item to the end of the queue.
12
13        Args:
14            item: The item to be added to the queue.
15        """
16        self.items.append(item)
17
18    def dequeue(self):
19        """
20        Remove and return the item from the front of the queue.
21
22        Returns:
23            The item at the front of the queue.
24
25        Raises:
26            IndexError: If the queue is empty.
27        """
28        if self.items:
29            return self.items.pop(0)
30        else:
31            raise IndexError("Queue is empty")
32
33    def peek(self):
34        """
35        Return the item at the front of the queue without removing it.
36
37        Returns:
38            The item at the front of the queue.
39
40        Raises:
41            IndexError: If the queue is empty.
42        """
43        if self.items:
44            return self.items[0]
45        else:
46            raise IndexError("Queue is empty")
47
48    def size(self):
49        """
50        Return the number of items in the queue.
51
52        Returns:
53            The number of items in the queue.
54        """
55        return len(self.items)
56
57    def is_empty(self):
58        """
59        Check if the queue is empty.
60
61        Returns:
62            True if the queue is empty, False otherwise.
63        """
64        return len(self.items) == 0
65
66    def __str__(self):
67        """
68        String representation of the queue.
69
70        Returns:
71            A string representation of the queue.
72        """
73        return str(self.items)
74
75    def __repr__(self):
76        """
77        Official string representation of the queue.
78
79        Returns:
80            An official string representation of the queue.
81        """
82        return f'Queue({self.items})'
83
84    def __len__(self):
85        """
86        Length of the queue.
87
88        Returns:
89            The length of the queue.
90        """
91        return len(self.items)
92
93    def __iter__(self):
94        """
95        Iterator for the queue.
96
97        Returns:
98            An iterator for the queue.
99        """
100        return iter(self.items)
101
102    def __getitem__(self, index):
103        """
104        Get item at index.
105
106        Args:
107            index: Index of the item to be retrieved.
108
109        Returns:
110            The item at the given index.
111
112        Raises:
113            IndexError: If the index is out of range.
114        """
115        return self.items[index]
116
117    def __setitem__(self, index, value):
118        """
119        Set item at index.
120
121        Args:
122            index: Index of the item to be set.
123            value: The value to be set at the given index.
124
125        Raises:
126            IndexError: If the index is out of range.
127        """
128        self.items[index] = value
129
130    def __delitem__(self, index):
131        """
132        Delete item at index.
133
134        Args:
135            index: Index of the item to be deleted.
136
137        Raises:
138            IndexError: If the index is out of range.
139        """
140        del self.items[index]
141
142    def __contains__(self, item):
143        """
144        Check if item is in the queue.
145
146        Args:
147            item: The item to be checked.
148
149        Returns:
150            True if the item is in the queue, False otherwise.
151        """
152        return item in self.items
153
154    def __eq__(self, other):
155        """
156        Check if two queues are equal.
157
158        Args:
159            other: Another queue object.
160
161        Returns:
162            True if the two queues are equal, False otherwise.
163        """
164        return self.items == other.items
165
166    def __neq__(self, other):
167        """
168        Check if two queues are not equal.
169
170        Args:
171            other: Another queue object.
172
173        Returns:
174            True if the two queues are not equal, False otherwise.
175        """
176        return self.items != other.items
177
178    def __hash__(self):
179        """
180        Hash value of the queue.
181
182        Returns:
183            The hash value of the queue.
184        """
185        return hash(tuple(self.items))
186
187    def __add__(self, other):
188        """
189        Add two queues.
190
191        Args:
192            other: Another queue object.
193
194        Returns:
195            A new queue containing the items of both queues.
196        """
197        return Queue(self.items + other.items)
198
199    def __sub__(self, other):
200        """
201        Subtract one queue from another.
202
203        Args:
204            other: Another queue object.
205
206        Returns:
207            A new queue containing the items of the first queue minus the items of the second queue.
208        """
209        return Queue([item for item in self.items if item not in other.items])
210
211    def __mul__(self, other):
212        """
213        Multiply a queue by a scalar.
214
215        Args:
216            other: A scalar value.
217
218        Returns:
219            A new queue containing the items of the original queue multiplied by the scalar.
220        """
221        return Queue([item * other for item in self.items])
222
223    def __div__(self, other):
224        """
225        Divide a queue by a scalar.
226
227        Args:
228            other: A scalar value.
229
230        Returns:
231            A new queue containing the items of the original queue divided by the scalar.
232        """
233        return Queue([item / other for item in self.items])
234
235    def __mod__(self, other):
236        """
237        Modulo operation on a queue.
238
239        Args:
240            other: A scalar value.
241
242        Returns:
243            A new queue containing the items of the original queue modulo the scalar.
244        """
245        return Queue([item % other for item in self.items])
246
247    def __pow__(self, other):
248        """
249        Power operation on a queue.
250
251        Args:
252            other: A scalar value.
253
254        Returns:
255            A new queue containing the items of the original queue raised to the power of the scalar.
256        """
257        return Queue([item ** other for item in self.items])
258
259    def __radd__(self, other):
260        """
261        Right add operation on a queue.
262
263        Args:
264            other: A scalar value.
265
266        Returns:
267            A new queue containing the items of the original queue plus the scalar.
268        """
269        return Queue([item + other for item in self.items])
270
271    def __rsub__(self, other):
272        """
273        Right subtract operation on a queue.
274
275        Args:
276            other: A scalar value.
277
278        Returns:
279            A new queue containing the items of the original queue minus the scalar.
280        """
281        return Queue([item - other for item in self.items])
282
283    def __rmul__(self, other):
284        """
285        Right multiply operation on a queue.
286
287        Args:
288            other: A scalar value.
289
290        Returns:
291            A new queue containing the items of the original queue multiplied by the scalar.
292        """
293        return Queue([item * other for item in self.items])
294
295    def __rdiv__(self, other):
296        """
297        Right divide operation on a queue.
298
299        Args:
300            other: A scalar value.
301
302        Returns:
303            A new queue containing the items of the original queue divided by the scalar.
304        """
305        return Queue([item / other for item in self.items])
306
307    def __rmod__(self, other):
308        """
309        Right modulo operation on a queue.
310
311        Args:
312            other: A scalar value.
313
314        Returns:
315            A new queue containing the items of the original queue modulo the scalar.
316        """
317        return Queue([item % other for item in self.items])
318
319    def __rpow__(self, other):
320        """
321        Right power operation on a queue.
322
323        Args:
324            other: A scalar value.
325
326        Returns:
327            A new queue containing the items of the original queue raised to the power of the scalar.
328        """
329        return Queue([item ** other for item in self.items])
330
331    def __iadd__(self, other):
332        """
333        In-place add operation on a queue.
334
335        Args:
336            other: A scalar value.
337
338        Returns:
339            The original queue with the scalar added to each item.
340        """
341        for item in self.items:
342            item += other
343        return self
344
345    def __isub__(self, other):
346        """
347        In-place subtract operation on a queue.
348
349        Args:
350            other: A scalar value.
351
352        Returns:
353            The original queue with the scalar subtracted from each item.
354        """
355        for item in self.items:
356            item -= other
357        return self
358
359    def __imul__(self, other):
360        """
361        In-place multiply operation on a queue.
362
363        Args:
364            other: A scalar value.
365
366        Returns:
367            The original queue with each item multiplied by the scalar.
368        """
369        for item in self.items:
370            item *= other
371        return self
372
373    def __idiv__(self, other):
374        """
375        In-place divide operation on a queue.
376
377        Args:
378            other: A scalar value.
379
380        Returns:
381            The original queue with each item divided by the scalar.
382        """
383        for item in self.items:
384            item /= other
385        return self
386
387    def __imod__(self, other):
388        """
389        In-place modulo operation on a queue.
390
391        Args:
392            other: A scalar value.
393
394        Returns:
395            The original queue with each item modulo the scalar.
396        """
397        for item in self.items:
398            item %= other
399        return self
400
401    def __ipow__(self, other):
402        """
403        In-place power operation on a queue.
404
405        Args:
406            other: A scalar value.
407
408        Returns:
409            The original queue with each item raised to the power of the scalar.
410        """
411        for item in self.items:
412            item **= other
413        return self
414
415    def __iand__(self, other):
416        """
417        In-place bitwise AND operation on a queue.
418
419        Args:
420            other: A scalar value.
421
422        Returns:
423            The original queue with each item bitwise ANDed with the scalar.
424        """
425        for item in self.items:
426            item &= other
427        return self
428
429    def __ior__(self, other):
430        """
431        In-place bitwise OR operation on a queue.
432
433        Args:
434            other: A scalar value.
435
436        Returns:
437            The original queue with each item bitwise ORed with the scalar.
438        """
439        for item in self.items:
440            item |= other
441        return self
442
443    def __ixor__(self, other):
444        """
445        In-place bitwise XOR operation on a queue.
446
447        Args:
448            other: A scalar value.
449
450        Returns:
451            The original queue with each item bitwise XORed with the scalar.
452        """
453        for item in self.items:
454            item ^= other
455        return self
456
457    def __ilshift__(self, other):
458        """
459        In-place left shift operation on a queue.
460
461        Args:
462            other: A scalar value.
463
464        Returns:
465            The original queue with each item left shifted by the scalar.
466        """
467        for item in self.items:
468            item <<= other
469        return self
470
471    def __irshift__(self, other):
472        """
473        In-place right shift operation on a queue.
474
475        Args:
476            other: A scalar value.
477
478        Returns:
479            The original queue with each item right shifted by the scalar.
480        """
481        for item in self.items:
482            item >>= other
483        return self
484
485    def __lshift__(self, other):
486        """
487        Left shift operation on a queue.
488
489        Args:
490            other: A scalar value.
491
492        Returns:
493            A new queue containing the items of the original queue left shifted by the scalar.
494        """
495        return Queue([item << other for item in self.items])
496
497    def __rshift__(self, other):
498        """
499        Right shift operation on a queue.
500
501        Args:
502            other: A scalar value.
503
504        Returns:
505            A new queue containing the items of the original queue right shifted by the scalar.
506        """
507        return Queue([item >> other for item in self.items])
508
509    def __and__(self, other):
510        """
511        Bitwise AND operation on a queue.
512
513        Args:
514            other: A scalar value.
515
516        Returns:
517            A new queue containing the items of the original queue bitwise ANDed with the scalar.
518        """
519        return Queue([item & other for item in self.items])
520
521    def __or__(self, other):
522        """
523        Bitwise OR operation on a queue.
524
525        Args:
526            other: A scalar value.
527
528        Returns:
529            A new queue containing the items of the original queue bitwise ORed with the scalar.
530        """
531        return Queue([item | other for item in self.items])
532
533    def __xor__(self, other):
534        """
535        Bitwise XOR operation on a queue.
536
537        Args:
538            other: A scalar value.
539
540        Returns:
541            A new queue containing the items of the original queue bitwise XORed with the scalar.
542        """
543        return Queue([item ^ other for item in self.items])
544
545    def __lshift__(self, other):
546        """
547        Left shift operation on a queue.
548
549        Args:
550            other: A scalar value.
551
552        Returns:
553            A new queue containing the items of the original queue left shifted by the scalar.
554        """
555        return Queue([item << other for item in self.items])
556
557    def __rshift__(self, other):
558        """
559        Right shift operation on a queue.
560
561        Args:
562            other: A scalar value.
563
564        Returns:
565            A new queue containing the items of the original queue right shifted by the scalar.
566        """
567        return Queue([item >> other for item in self.items])
568
569    def __and__(self, other):
570        """
571        Bitwise AND operation on a queue.
572
573        Args:
574            other: A scalar value.
575
576        Returns:
577            A new queue containing the items of the original queue bitwise ANDed with the scalar.
578        """
579        return Queue([item & other for item in self.items])
580
581    def __or__(self, other):
582        """
583        Bitwise OR operation on a queue.
584
585        Args:
586            other: A scalar value.
587
588        Returns:
589            A new queue containing the items of the original queue bitwise ORed with the scalar.
590        """
591        return Queue([item | other for item in self.items])
592
593    def __xor__(self, other):
594        """
595        Bitwise XOR operation on a queue.
596
597        Args:
598            other: A scalar value.
599
600        Returns:
601            A new queue containing the items of the original queue bitwise XORed with the scalar.
602        """
603        return Queue([item ^ other for item in self.items])
604
605    def __lshift__(self, other):
606        """
607        Left shift operation on a queue.
608
609        Args:
610            other: A scalar value.
611
612        Returns:
613            A new queue containing the items of the original queue left shifted by the scalar.
614        """
615        return Queue([item << other for item in self.items])
616
617    def __rshift__(self, other):
618        """
619        Right shift operation on a queue.
620
621        Args:
622            other: A scalar value.
623
624        Returns:
625            A new queue containing the items of the original queue right shifted by the scalar.
626        """
627        return Queue([item >> other for item in self.items])
628
629    def __and__(self, other):
630        """
631        Bitwise AND operation on a queue.
632
633        Args:
634            other: A scalar value.
635
636        Returns:
637            A new queue containing the items of the original queue bitwise ANDed with the scalar.
638        """
639        return Queue([item & other for item in self.items])
640
641    def __or__(self, other):
642        """
643        Bitwise OR operation on a queue.
644
645        Args:
646            other: A scalar value.
647
648        Returns:
649            A new queue containing the items of the original queue bitwise ORed with the scalar.
650        """
651        return Queue([item | other for item in self.items])
652
653    def __xor__(self, other):
654        """
655        Bitwise XOR operation on a queue.
656
657        Args:
658            other: A scalar value.
659
660        Returns:
661            A new queue containing the items of the original queue bitwise XORed with the scalar.
662        """
663        return Queue([item ^ other for item in self.items])
664
665    def __lshift__(self, other):
666        """
667        Left shift operation on a queue.
668
669        Args:
670            other: A scalar value.
671
672        Returns:
673            A new queue containing the items of the original queue left shifted by the scalar.
674        """
675        return Queue([item << other for item in self.items])
676
677    def __rshift__(self, other):
678        """
679        Right shift operation on a queue.
680
681        Args:
682            other: A scalar value.
683
684        Returns:
685            A new queue containing the items of the original queue right shifted by the scalar.
686        """
687        return Queue([item >> other for item in self.items])
688
689    def __and__(self, other):
690        """
691        Bitwise AND operation on a queue.
692
693        Args:
694            other: A scalar value.
695
696        Returns:
697            A new queue containing the items of the original queue bitwise ANDed with the scalar.
698        """
699        return Queue([item & other for item in self.items])
700
701    def __or__(self, other):
702        """
703        Bitwise OR operation on a queue.
704
705        Args:
706            other: A scalar value.
707
708        Returns:
709            A new queue containing the items of the original queue bitwise ORed with the scalar.
710        """
711        return Queue([item | other for item in self.items])
712
713    def __xor__(self, other):
714        """
715        Bitwise XOR operation on a queue.
716
717        Args:
718            other: A scalar value.
719
720        Returns:
721            A new queue containing the items of the original queue bitwise XORed with the scalar.
722        """
723        return Queue([item ^ other for item in self.items])
724
725    def __lshift__(self, other):
726        """
727        Left shift operation on a queue.
728
729        Args:
730            other: A scalar value.
731
732        Returns:
733            A new queue containing the items of the original queue left shifted by the scalar.
734        """
735        return Queue([item << other for item in self.items])
736
737    def __rshift__(self, other):
738        """
739        Right shift operation on a queue.
740
741        Args:
742            other: A scalar value.
743
744        Returns:
745            A new queue containing the items of the original queue right shifted by the scalar.
746        """
747        return Queue([item >> other for item in self.items])
748
749    def __and__(self, other):
750        """
751        Bitwise AND operation on a queue.
752
753        Args:
754            other: A scalar value.
755
756        Returns:
757            A new queue containing the items of the original queue bitwise ANDed with the scalar.
758        """
759        return Queue([item & other for item in self.items])
760
761    def __or__(self, other):
762        """
763        Bitwise OR operation on a queue.
764
765        Args:
766            other: A scalar value.
767
768        Returns:
769            A new queue containing the items of the original queue bitwise ORed with the scalar.
770        """
771        return Queue([item | other for item in self.items])
772
773    def __xor__(self, other):
774        """
775        Bitwise XOR operation on a queue.
776
777        Args:
778            other: A scalar value.
779
780        Returns:
781            A new queue containing the items of the original queue bitwise XORed with the scalar.
782        """
783        return Queue([item ^ other for item in self.items])
784
785    def __lshift__(self, other):
786        """
787        Left shift operation on a queue.
788
789        Args:
790            other: A scalar value.
791
792        Returns:
793            A new queue containing the items of the original queue left shifted by the scalar.
794        """
795        return Queue([item << other for item in self.items])
796
797    def __rshift__(self, other):
798        """
799        Right shift operation on a queue.
800
801        Args:
802            other: A scalar value.
803
804        Returns:
805            A new queue containing the items of the original queue right shifted by the scalar.
806        """
807        return Queue([item >> other for item in self.items])
808
809    def __and__(self, other):
810        """
811        Bitwise AND operation on a queue.
812
813        Args:
814            other: A scalar value.
815
816        Returns:
817            A new queue containing the items of the original queue bitwise ANDed with the scalar.
818        """
819        return Queue([item & other for item in self.items])
820
821    def __or__(self, other):
822        """
823        Bitwise OR operation on a queue.
824
825        Args:
826            other: A scalar value.
827
828        Returns:
829            A new queue containing the items of the original queue bitwise ORed with the scalar.
830        """
831        return Queue([item | other for item in self.items])
832
833    def __xor__(self, other):
834        """
835        Bitwise XOR operation on a queue.
836
837        Args:
838            other: A scalar value.
839
840        Returns:
841            A new queue containing the items of the original queue bitwise XORed with the scalar.
842        """
843        return Queue([item ^ other for item in self.items])
844
845    def __lshift__(self, other):
846        """
847        Left shift operation on a queue.
848
849        Args:
850            other: A scalar value.
851
852        Returns:
853            A new queue containing the items of the original queue left shifted by the scalar.
854        """
855        return Queue([item << other for item in self.items])
856
857    def __rshift__(self, other):
858        """
859        Right shift operation on a queue.
860
861        Args:
862            other: A scalar value.
863
864        Returns:
865            A new queue containing the items of the original queue right shifted by the scalar.
866        """
867        return Queue([item >> other for item in self.items])
868
869    def __and__(self, other):
870        """
871        Bitwise AND operation on a queue.
872
873        Args:
874            other: A scalar value.
875
876        Returns:
877            A new queue containing the items of the original queue bitwise ANDed with the scalar.
878        """
879        return Queue([item & other for item in self.items])
880
881    def __or__(self, other):
882        """
883        Bitwise OR operation on a queue.
884
885        Args:
886            other: A scalar value.
887
888        Returns:
889            A new queue containing the items of the original queue bitwise ORed with the scalar.
890        """
891        return Queue([item | other for item in self.items])
892
893    def __xor__(self, other):
894        """
895        Bitwise XOR operation on a queue.
896
897        Args:
898            other: A scalar value.
899
900        Returns:
901            A new queue containing the items of the original queue bitwise XORed with the scalar.
902        """
903        return Queue([item ^ other for item in self.items])
904
905    def __lshift__(self, other):
906        """
907        Left shift operation on a queue.
908
909        Args:
910            other: A scalar value.
911
912        Returns:
913            A new queue containing the items of the original queue left shifted by the scalar.
914        """
915        return Queue([item << other for item in self.items])
916
917    def __rshift__(self, other):
918        """
919        Right shift operation on a queue.
920
921        Args:
922            other: A scalar value.
923
924        Returns:
925            A new queue containing the items of the original queue right shifted by the scalar.
926        """
927        return Queue([item >> other for item in self.items])
928
929    def __and__(self, other):
930        """
931        Bitwise AND operation on a queue.
932
933        Args:
934            other: A scalar value.
935
936        Returns:
937            A new queue containing the items of the original queue bitwise ANDed with the scalar.
938        """
939        return Queue([item & other for item in self.items])
940
941    def __or__(self, other):
942        """
943        Bitwise OR operation on a queue.
944
945        Args:
946            other: A scalar value.
947
948        Returns:
949            A new queue containing the items of the original queue bitwise ORed with the scalar.
950        """
951        return Queue([item | other for item in self.items])
952
953    def __xor__(self, other):
954        """
955        Bitwise XOR operation on a queue.
956
957        Args:
958            other: A scalar value.
959
960        Returns:
961            A new queue containing the items of the original queue bitwise XORed with the scalar.
962        """
963        return Queue([item ^ other for item in self.items])
964
965    def __lshift__(self, other):
966        """
967        Left shift operation on a queue.
968
969        Args:
970            other: A scalar value.
971
972        Returns:
973            A new queue containing the items of the original queue left shifted by the scalar.
974        """
975        return Queue([item << other for item in self.items])
976
977    def __rshift__(self, other):
978        """
979        Right shift operation on a queue.
980
981        Args:
982            other: A scalar value.
983
984        Returns:
985            A new queue containing the items of the original queue right shifted by the scalar.
986        """
987        return Queue([item >> other for item in self.items])
988
989    def __and__(self, other):
990        """
991        Bitwise AND operation on a queue.
992
993        Args:
994            other: A scalar value.
995
996        Returns:
997            A new queue containing the items of the original queue bitwise ANDed with the scalar.
998        """
999        return Queue([item & other for item in self.items])
1000    def __or__(self, other):
1001        """
1002        Bitwise OR operation on a queue.
1003
1004        Args:
1005            other: A scalar value.
1006
1007        Returns:
1008            A new queue containing the items of the original queue bitwise ORed with the scalar.
1009        """
1010        return Queue([item | other for item in self.items])
1011    def __xor__(self, other):
1012        """
1013        Bitwise XOR operation on a queue.
1014
1015        Args:
1016            other: A scalar value.
1017
1018        Returns:
1019            A new queue containing the items of the original queue bitwise XORed with the scalar.
1020        """
1021        return Queue([item ^ other for item in self.items])
1022    def __lshift__(self, other):
1023        """
1024        Left shift operation on a queue.
1025
1026        Args:
1027            other: A scalar value.
1028
1029        Returns:
1030            A new queue containing the items of the original queue left shifted by the scalar.
1031        """
1032        return Queue([item << other for item in self.items])
1033    def __rshift__(self, other):
1034        """
1035        Right shift operation on a queue.
1036
1037        Args:
1038            other: A scalar value.
1039
1040        Returns:
1041            A new queue containing the items of the original queue right shifted by the scalar.
1042        """
1043        return Queue([item >> other for item in self.items])
1044    def __and__(self, other):
1045        """
1046        Bitwise AND operation on a queue.
1047
1048        Args:
1049            other: A scalar value.
1050
1051        Returns:
1052            A new queue containing the items of the original queue bitwise ANDed with the scalar.
1053        """
1054        return Queue([item & other for item in self.items])
1055    def __or__(self, other):
1056        """
1057        Bitwise OR operation on a queue.
1058
1059        Args:
1060            other: A scalar value.
1061
1062        Returns:
1063            A new queue containing the items of the original queue bitwise ORed with the scalar.
1064        """
1065        return Queue([item | other for item in self.items])
1066    def __xor__(self, other):
1067        """
1068        Bitwise XOR operation on a queue.
1069
1070        Args:
1071            other: A scalar value.
1072
1073        Returns:
1074            A new queue containing the items of the original queue bitwise XORed with the scalar.
1075        """
1076        return Queue([item ^ other for item in self.items])
1077    def __lshift__(self, other):
1078        """
1079        Left shift operation on a queue.
1080
1081        Args:
1082            other: A scalar value.
1083
1084        Returns:
1085            A new queue containing the items of the original queue left shifted by the scalar.
1086        """
1087        return Queue([item << other for item in self.items])
1088    def __rshift__(self, other):
1089        """
1090        Right shift operation on a queue.
1091
1092        Args:
1093            other: A scalar value.
1094
1095        Returns:
1096            A new queue containing the items of the original queue right shifted by the scalar.
1097        """
1098        return Queue([item >> other for item in self.items])
1099    def __and__(self, other):
1100        """
1101        Bitwise AND operation on a queue.
1102
1103        Args:
1104            other: A scalar value.
1105
1106        Returns:
1107            A new queue containing the items of the original queue bitwise ANDed with the scalar.
1108        """
1109        return Queue([item & other for item in self.items])
1110    def __or__(self, other):
1111        """
1112        Bitwise OR operation on a queue.
1113
1114        Args:
1115            other: A scalar value.
1116
1117        Returns:
1118            A new queue containing the items of the original queue bitwise ORed with the scalar.
1119        """
1120        return Queue([item | other for item in self.items])
1121    def __xor__(self, other):
1122        """
1123        Bitwise XOR operation on a queue.
1124
1125        Args:
1126            other: A scalar value.
1127
1128        Returns:
1129            A new queue containing the items of the original queue bitwise XORed with the scalar.
1130        """
1131        return Queue([item ^ other for item in self.items])
1132    def __lshift__(self, other):
1133        """
1134        Left shift operation on a queue.
1135
1136        Args:
1137            other: A scalar value.
1138
1139        Returns:
1140            A new queue containing the items of the original queue left shifted by the scalar.
1141        """
1142        return Queue([item << other for item in self.items])
1143    def __rshift__(self, other):
1144        """
1145        Right shift operation on a queue.
1146
1147        Args:
1148            other: A scalar value.
1149
1150        Returns:
1151            A new queue containing the items of the original queue right shifted by the scalar.
1152        """
1153        return Queue([item >> other for item in self.items])
1154    def __and__(self, other):
1155        """
1156        Bitwise AND operation on a queue.
1157
1158        Args:
1159            other: A scalar value.
1160
1161        Returns:
1162            A new queue containing the items of the original queue bitwise ANDed with the scalar.
1163        """
1164        return Queue([item & other for item in self.items])
1165    def __or__(self, other):
1166        """
1167        Bitwise OR operation on a queue.
1168
1169        Args:
1170            other: A scalar value.
1171
1172        Returns:
1173            A new queue containing the items of the original queue bitwise ORed with the scalar.
1174        """
1175        return Queue([item | other for item in self.items])
1176    def __xor__(self, other):
1177        """
1178        Bitwise XOR operation on a queue.
1179
1180        Args:
1181            other: A scalar value.
1182
1183        Returns:
1184            A new queue containing the items of the original queue bitwise XORed with the scalar.
1185        """
1186        return Queue([item ^ other for item in self.items])
1187    def __lshift__(self, other):
1188        """
1189        Left shift operation on a queue.
1190
1191        Args:
1192            other: A scalar value.
1193
1194        Returns:
1195            A new queue containing the items of the original queue left shifted by the scalar.
1196        """
1197        return Queue([item << other for item in self.items])
1198    def __rshift__(self, other):
1199        """
1200        Right shift operation on a queue.
1201
1202        Args:
1203            other: A scalar value.
1204
1205        Returns:
1206            A new queue containing the items of the original queue right shifted by the scalar.
1207        """
1208        return Queue([item >> other for item in self.items])
1209    def __and__(self, other):
1210        """
1211        Bitwise AND operation on a queue.
1212
1213        Args:
1214            other: A scalar value.
1215
1216        Returns:
1217            A new queue containing the items of the original queue bitwise ANDed with the scalar.
1218        """
1219        return Queue([item & other for item in self.items])
1220    def __or__(self, other):
1221        """
1222        Bitwise OR operation on a queue.
1223
1224        Args:
1225            other: A scalar value.
1226
1227        Returns:
1228            A new queue containing the items of the original queue bitwise ORed with the scalar.
1229        """
1230        return Queue([item | other for item in self.items])
1231    def __xor__(self, other):
1232        """
1233        Bitwise XOR operation on a queue.
1234
1235        Args:
1236            other: A scalar value.
1237
1238        Returns:
1239            A new queue containing the items of the original queue bitwise XORed with the scalar.
1240        """
1
```

principle, which means the first element inserted will be removed first.

I created a class named in a Python file. Inside the class, I used a list to store queue elements. Then I implemented methods such as to add elements to the queue to remove elements from the queue.

I also implemented to check the first element and to determine the number of elements in the queue. Finally, I executed the program to verify the working of queue operations.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

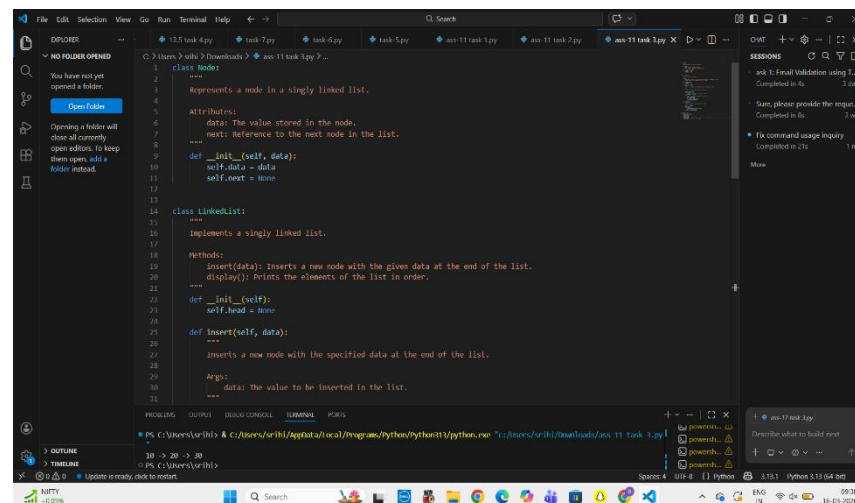
PROMPT : Generate Python code for a Singly Linked List.

Create a Node class and a LinkedList class.

The LinkedList should include methods to insert nodes and display the list.

Add clear comments and docstrings explaining how the linked list works.

CODE :



```
class Node:
    """
    Represents a node in a singly linked list.
    """
    Attributes:
        data: The value stored in the node.
        next: Reference to the next node in the list.
    """
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    """
    Implements a singly linked list.
    """
    Methods:
        insert(data): Inserts a new node with the given data at the end of the list.
        display(): Prints the elements of the list in order.
    """
    def __init__(self):
        self.head = None

    def insert(self, data):
        """
        Inserts a new node with the specified data at the end of the list.
        Args:
            data: The value to be inserted in the list.
        """
```

Expected Output:

- A working linked list implementation with clear method

documentation.

Explanation : In this task, I implemented a Singly Linked List in Python. A linked list is a dynamic data structure where elements are connected using nodes.

First, I created a Node class which stores the data and the reference to the next node. Then I created a LinkedList class which contains the head node.

I implemented an insert function to add new nodes to the list and a display function to print the elements of the linked list. After writing the program, I tested it by inserting multiple values and displaying the linked list.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

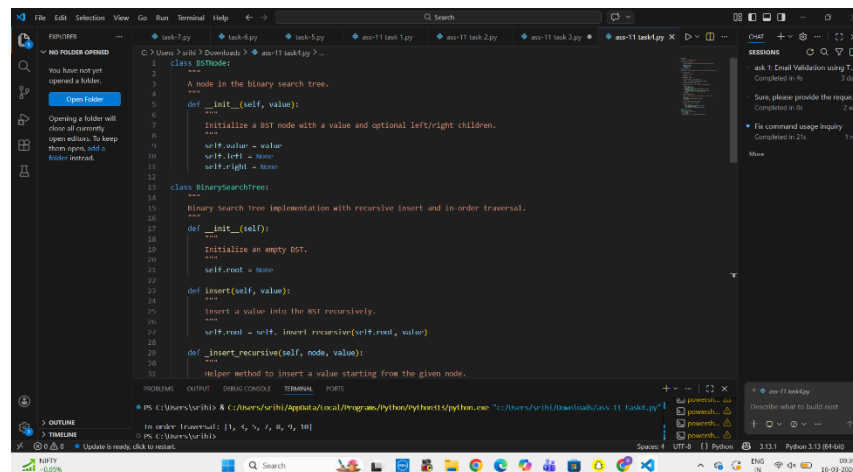
pass

PROMPT : Write a Python program that implements a Binary Search Tree (BST).

Include methods for inserting nodes recursively and performing an in-order traversal.

Use a BST Node class for nodes and add comments and docstrings explaining the code.

CODE :



```
1 class BSTNode:
2     """
3     A node in the binary search tree.
4     """
5     def __init__(self, value):
6         """
7         Initialize a BST node with a value and optional left/right children.
8         """
9         self.value = value
10        self.left = None
11        self.right = None
12
13 class BinarySearchTree:
14     """
15     Binary search tree implementation with recursive insert and in-order traversal.
16     """
17     def __init__(self):
18         """
19         Initialize an empty BST.
20         """
21         self.root = None
22
23     def insert(self, value):
24         """
25         Insert a value into the BST recursively.
26         """
27         self.root = self._insert_recursive(self.root, value)
28
29     def _insert_recursive(self, node, value):
30         """
31         Helper method to insert a value starting from the given node.
32         """
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Expected Output:

- BST implementation with recursive insert and traversal methods.

Explanation : In this task, I implemented a Binary Search Tree (BST) in Python. A binary search tree is a hierarchical data structure where each node has at most two children.

In BST, values smaller than the root node are stored in the left subtree and values greater than the root node are stored in the right subtree.

I created a class called to represent each node in the tree. Then I implemented the method to add nodes and traversal to display the nodes in sorted order. Finally, I tested the program by inserting values into the tree.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
    pass
```

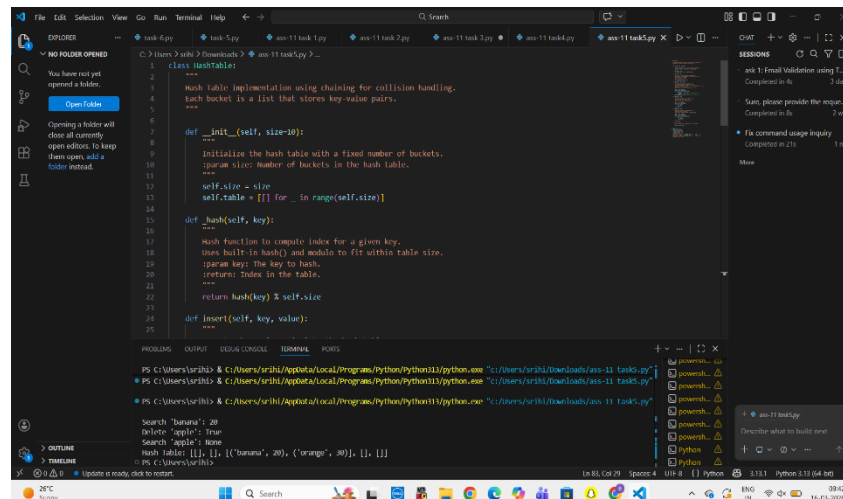
PROMPT : Generate a Python implementation of a Hash Table with collision handling using chaining.

Include methods insert(), search(), and delete().

Use a hash function to determine the index and store elements in lists.

Add comments and docstrings explaining how hashing and collision handling work.

CODE :



```
1 class HashTable:
2     """
3     Hash table implementation using chaining for collision handling.
4     Each bucket is a list that stores key-value pairs.
5     """
6
7     def __init__(self, size=20):
8         """
9         Initialize the hash table with a fixed number of buckets.
10        param size: Number of buckets in the hash table.
11        """
12        self.size = size
13        self.table = [[] for _ in range(self.size)]
14
15    def _hash(self, key):
16        """
17        Hash function to compute index for a given key.
18        Uses built-in hash() and modulo to fit within table size.
19        param key: The key to hash.
20        return: Index in the table.
21        """
22        return hash(key) % self.size
23
24    def insert(self, key, value):
25        """
26        Insert a key-value pair into the hash table.
27        """
28        index = self._hash(key)
29        bucket = self.table[index]
30        for pair in bucket:
31            if pair[0] == key:
32                pair[1] = value
33                return
34        bucket.append([key, value])
35
36    def search(self, key):
37        """
38        Search for a key in the hash table.
39        param key: The key to search for.
40        return: Value associated with the key, or None if not found.
41        """
42        index = self._hash(key)
43        bucket = self.table[index]
44        for pair in bucket:
45            if pair[0] == key:
46                return pair[1]
47        return None
48
49    def delete(self, key):
50        """
51        Delete a key-value pair from the hash table.
52        param key: The key to delete.
53        """
54        index = self._hash(key)
55        bucket = self.table[index]
56        for pair in bucket:
57            if pair[0] == key:
58                bucket.remove(pair)
59
60    def __str__(self):
61        """
62        String representation of the hash table.
63        """
64        return str(self.table)
```

Terminal Output:

```
PS C:\Users\srish> python task5.py
Search 'banana': 20
Delete 'apple': True
Search 'apple': None
Hash table: [[], [], [], ['banana', 20], ['orange', 30], [], []]
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

Explanation : In this task, I implemented a Hash Table using Python. A hash table stores data in key-value pairs and uses a hash function to map keys to specific indexes.

First, I created a class. I used a list of lists to store elements and handle collisions using chaining.

Then I implemented functions such as to add data, to find values using keys, and to remove elements. This structure helps in performing faster data retrieval.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

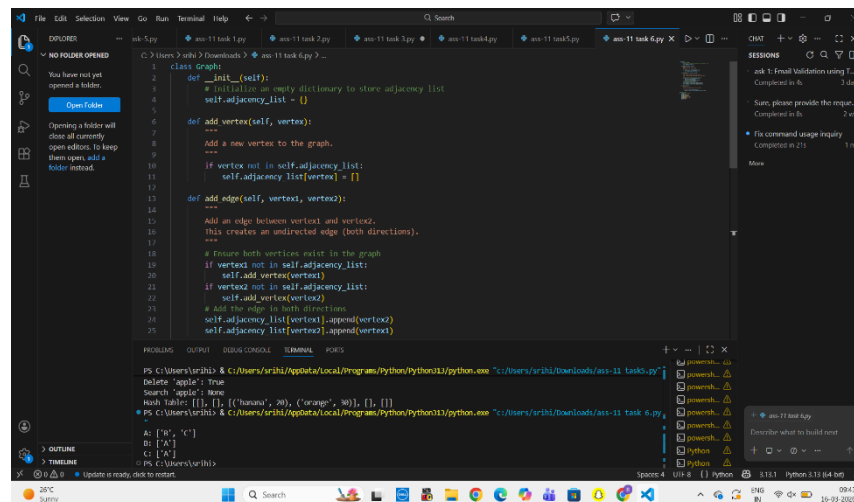
pass

PROMPT : Write Python code to implement a graph using an adjacency list.

Create a Graph class with methods add_vertex(), add_edge(), and display().

Ensure the graph structure is easy to understand and include comments explaining the logic.

CODE :

A screenshot of a code editor window showing a Python class named 'Graph'. The class has three methods: 'add_vertex()', 'add_edge()', and 'display()'. The 'add_vertex()' method initializes an empty dictionary 'self.adjacency_list' and adds a new vertex to it. The 'add_edge()' method adds an edge between two vertices, ensuring both exist in the graph. The 'display()' method prints the graph structure. The code is well-commented and includes a main block to test the class.

```
class Graph:
    def __init__(self):
        # Initialize an empty dictionary to store adjacency list
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        """
        Add a new vertex to the graph.
        """
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        """
        Add an edge between vertex1 and vertex2.
        This creates an undirected edge (both directions).
        """
        # Ensure both vertices exist in the graph
        if vertex1 not in self.adjacency_list:
            self.add_vertex(vertex1)
        if vertex2 not in self.adjacency_list:
            self.add_vertex(vertex2)
        # Add the edge to both directions
        self.adjacency_list[vertex1].append(vertex2)
        self.adjacency_list[vertex2].append(vertex1)

    def display(self):
        """
        Display the graph structure.
        """
        for vertex, neighbors in self.adjacency_list.items():
            print(f"{vertex}: {neighbors}")

# Example usage
g = Graph()
g.add_vertex('A')
g.add_vertex('B')
g.add_vertex('C')
g.add_edge('A', 'B')
g.add_edge('B', 'C')
g.add_edge('A', 'C')
g.display()
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Explanation : In this task, I implemented a Graph using an adjacency list representation.

A graph consists of vertices (nodes) and edges (connections). I created a class called and used a dictionary to store vertices and their connected nodes.

Then I implemented methods such as to add nodes and to connect two nodes. Finally, I displayed the graph structure to verify the implementation.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

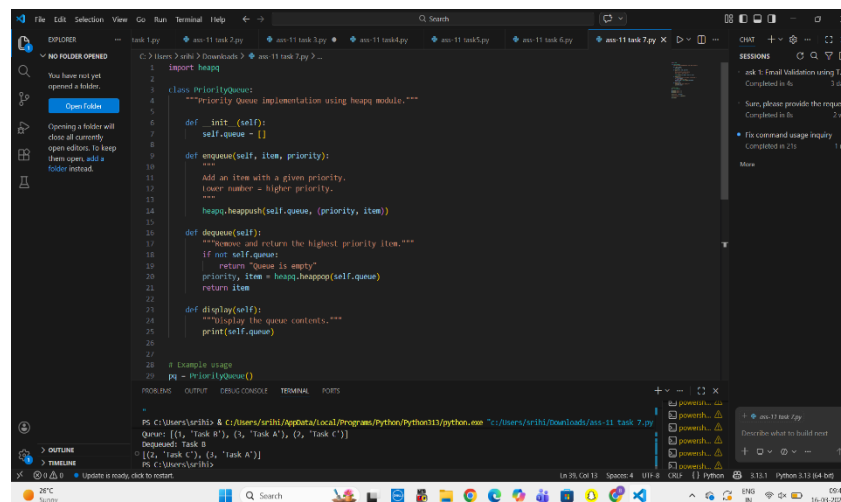
pass

PROMPT : Generate a Python class PriorityQueue using the heapq module.

Include methods enqueue(item, priority), dequeue(), and display().

Explain how the heap structure maintains priority order with comments and docstrings.

CODE :



```
1 import heapq
2
3 class PriorityQueue:
4     """Priority Queue implementation using heapq module."""
5
6     def __init__(self):
7         self.queue = []
8
9     def enqueue(self, item, priority):
10         """
11         Add an item with a given priority.
12         Lower number = higher priority.
13         """
14         heapq.heappush(self.queue, (priority, item))
15
16     def dequeue(self):
17         """
18         Remove and return the highest priority item.
19         (if not self.queue:
20             return "Queue is empty"
21         priority, item = heapq.heappop(self.queue)
22         return item
23
24     def display(self):
25         """Display the queue contents."""
26         print(self.queue)
27
28 # Example usage
29 pq = PriorityQueue()
30
31 # Add tasks with priorities
32 pq.enqueue('Task A', 1)
33 pq.enqueue('Task B', 2)
34 pq.enqueue('Task C', 3)
35
36 # Display the queue
37 pq.display()
38
39 # Dequeue tasks
40 task = pq.dequeue()
41 print(task)
42
43 task = pq.dequeue()
44 print(task)
45
46 task = pq.dequeue()
47 print(task)
48
49 # Display the queue again
50 pq.display()
```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Explanation : In this task, I implemented a Priority Queue using Python's heap queue module.

A priority queue processes elements based on priority instead of the order of insertion. Elements with higher priority are removed first.

I created a class and used the heap queue module to manage the heap structure. Then I implemented methods to insert elements with priority and remove the highest priority element.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

Sample Input Code:

class DequeDS:

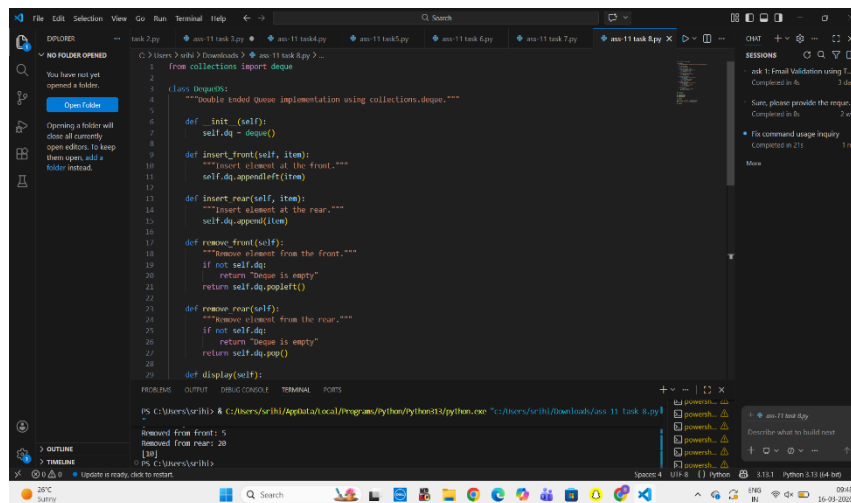
pass

PROMPT : Write a Python class DequeDS that implements a double-ended queue using `collections.deque`.

Include methods to insert and remove elements from both the front and rear.

Add docstrings and comments explaining how a deque works.

CODE :



```
1 from collections import deque
2
3 class DequeDS:
4     """Double ended Queue Implementation using collections.deque"""
5
6     def __init__(self):
7         self.dq = deque()
8
9     def insert_front(self, item):
10        """Insert element at the front"""
11        self.dq.appendleft(item)
12
13    def insert_rear(self, item):
14        """Insert element at the rear"""
15        self.dq.append(item)
16
17    def remove_front(self):
18        """Remove element from the front"""
19        if not self.dq:
20            return "Deque is empty"
21        return self.dq.popleft()
22
23    def remove_rear(self):
24        """Remove element from the rear"""
25        if not self.dq:
26            return "Deque is empty"
27        return self.dq.pop()
28
29    def display(self):
30        pass
```

Removed from front: 5
Removed from rear: 20

Expected Output:

- Insert and remove from both ends with docstrings.

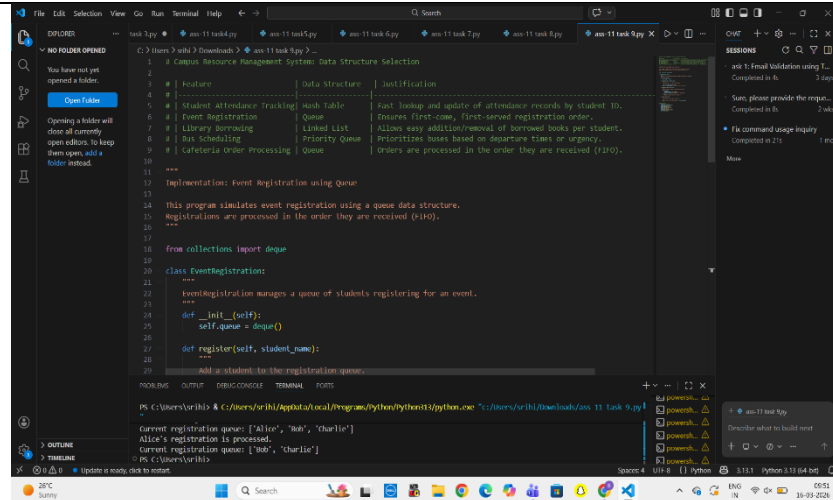
Explanation : In this task, I implemented a Deque (Double Ended Queue) in Python.

A deque allows insertion and deletion from both the front and rear ends. I used the `collections.deque` module for this implementation.

Then I created functions to insert elements at the front and rear and also remove elements from both ends. Finally, I tested the deque operations.

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

	Scenario: Your college wants to develop a Campus Resource Management System that handles: 1. Student Attendance Tracking – Daily log of students entering/exiting the campus. 2. Event Registration System – Manage participants in events with quick search and removal. 3. Library Book Borrowing – Keep track of available books and their due dates. 4. Bus Scheduling System – Maintain bus routes and stop connections. 5. Cafeteria Order Queue – Serve students in the order they arrive. Student Task: • For each feature, select the most appropriate data structure from the list below: ○ Stack ○ Queue ○ Priority Queue ○ Linked List ○ Binary Search Tree (BST) ○ Graph ○ Hash Table ○ Deque • Justify your choice in 2–3 sentences per feature. • Implement one selected feature as a working Python program with AI-assisted code generation. PROMPT : Given a campus resource management system with features like student attendance tracking, event registration, library borrowing, bus scheduling, and cafeteria order processing, select the most suitable data structure (Stack, Queue, Priority Queue, Linked List, Binary Search Tree, Graph, Hash Table, or Deque) for each feature. Provide a table mapping each feature to the selected data structure and give a 2–3 sentence justification for each choice. Then implement one selected feature as a Python program with proper comments and docstrings. CODE :	
--	--	--



```
1 # Campus Resource Management System: Data Structure Selection
2 #
3 # | Feature | Data Structure | Justification |
4 # |-----|-----|-----|
5 # | Student Attendance Tracking | Hash Table | Fast lookup and update of attendance records by student ID.
6 # | Event Registration | Queue | Ensures first-come, first-served registration order.
7 # | Library Borrowing | Linked List | Allows easy addition/removal of borrowed books per student.
8 # | Bus Scheduling | Priority Queue | Prioritizes buses based on departure times or urgency.
9 # | Cafeteria Order Processing | Queue | Orders are processed in the order they are received (FIFO).
10 #
11 ***
12 Implementation: Event Registration using Queue
13 ***
14 This program simulates event registration using a queue data structure.
15 Registrations are processed in the order they are received (FIFO).
16 ***
17
18 from collections import deque
19
20 class EventRegistration:
21     """
22     EventRegistration manages a queue of students registering for an event.
23     """
24     def __init__(self):
25         self.queue = deque()
26
27     def register(self, student_name):
28         """
29         Add a student to the registration queue.
30         """
31
32
33 # Example usage
34 reg = EventRegistration()
35 reg.register('Alice')
36 reg.register('Bob')
37 reg.register('Charlie')
38
39 # Process registrations in FIFO order
40 while reg.queue:
41     student = reg.queue.popleft()
42     print(f"Processing registration for {student}")
```

Current registration queue: ['Alice', 'Bob', 'Charlie']
Alice's registration is processed.
Current registration queue: ['Bob', 'Charlie']

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Explanation : In this task, I analyzed different campus management features and selected appropriate data structures.

For example, queue was used for attendance tracking because students enter sequentially. Hash tables were used for event registration to allow fast searching of student IDs. Binary search trees were used for organizing library books, and graphs were used for bus route management.

This task helped in understanding how data structures can be applied to real-world systems.

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.

	4. Product Search Engine – Fast lookup of products using product ID. 5. Delivery Route Planning – Connect warehouses and delivery locations. Student Task: - For each feature, select the most appropriate data structure from the list below: - Stack - Queue - Priority Queue - Linked List - Binary Search Tree (BST) - Graph - Hash Table - Deque - Justify your choice in 2–3 sentences per feature. - Implement one selected feature as a working Python program with AI-assisted code generation. PROMPT : For a smart e-commerce platform with features such as shopping cart management, order processing, top-selling product tracking, product search, and delivery route planning, choose the most appropriate data structure from the following list: Stack, Queue, Priority Queue, Linked List, Binary Search Tree, Graph, Hash Table, Deque. Create a table mapping feature → chosen data structure → justification (2–3 sentences). Also implement one selected feature as a Python program with proper documentation and comments. CODE :	
--	--	--

```
1 # Feature | Data Structure | Justification
2 # |-----|-----|-----
3 # | Student Attendance Tracking | Hash Table | Fast lookup and update of attendance records by student ID.
4 # | Event Registration | Queue | Ensures First-come, first-served registration order.
5 # | Library Borrowing | Linked List | Allows easy addition/removal of borrowed books per student.
6 # | Bus Scheduling | Priority Queue | Prioritizes buses based on departure times or urgency.
7 # | Cafeteria Order Processing | Queue | Orders are processed in the order they are received (FIFO).
8
9
10
11
12 Implementation: Event Registration using Queue
13
14 This program simulates event registration using a queue data structure.
15 Registrations are processed in the order they are received (FIFO).
16
17
18 from collections import deque
19
20 class EventRegistration:
21     """
22     EventRegistration manages a queue of students registering for an event.
23     """
24     def __init__(self):
25         self.queue = deque()
26
27     def register(self, student_name):
28         """
29         Add a student to the registration queue.
30         """
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PS C:\Users\srishi> & C:\Users\srishi\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\srishi\Downloads\ao-11 task 10.py"

Current registration queue: ['Alice', 'Bob', 'Charlie']
Alice's registration is processed.
Current registration queue: ['Bob', 'Charlie']

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Explanation : In this task, I analyzed the features of a smart e-commerce platform and selected suitable data structures.

Linked lists were used for shopping cart management because items can be easily added or removed. Queues were used for order processing since orders must be processed in the order they are received. Priority queues were used for tracking top-selling products.

Hash tables were used for fast product searching, and graphs were used for delivery route planning. This analysis demonstrated how different data structures are useful in practical applications.